

Shipping the Kiosk to the Raspberry Pi

A step-by-step guide from "it works on the laptop" to a touchscreen kiosk that survives reboots, sends its messages, and backs itself up – with the traps to look out for at every step.

Raspberry Pi 4

Raspberry Pi OS 64-bit

Companion to the Team Manual

September 2026

1 What you're building

The Pi is the production machine. The laptop stays for development.

On the laptop you start the kiosk by hand. On the Pi, nothing may depend on a person:

systemd starts everything at boot, restarts the web app if it crashes, and runs the background workers on timers. This is the whole machine you are about to set up:

THE PI AT BOOT — EVERYTHING STARTS ITSELF

systemd

the Pi's service manager – starts everything below



maintenance-kiosk.service

unicorn serves the app on port 5050 – always on, auto-restarts

notifications.timer

every 5 min – sends queued
Discord / Chat / email

sheets-sync.timer

every 5 min – pushes queued
rows to Google Sheets

weekly-reminder.timer

Monday 12:00 Pi local time –
queues reminder emails

meanwhile, the desktop autostart opens the screen



Chromium in kiosk mode

full-screen on the 800 × 480 touchscreen → <http://127.0.0.1:5050>

Four files/folders on the Pi matter after setup – everything else is code from git:

PATH ON THE PI	WHAT IT IS	IN GIT?
<code>data/kiosk.db</code>	The real database – all history and settings	No (ignored) – never overwritten by updates
<code>data/diagnosis_tree.json</code>	The Diagnosis Assistant's tree	Yes – tracked. See section 7 before ever running <code>git pull</code>
<code>data/diagnosis_tree_backups/</code>	Automatic tree snapshots (last 25 saves)	No
<code>.env</code>	Email credentials and secrets	No (ignored) – created by hand on the Pi

2 Before you leave the laptop

The Pi gets its code from GitHub – not from your laptop's folder.

Pre-flight checklist (5 minutes, saves an evening)

- 1 Run the tests:

```
.venv/bin/python -m unittest discover tests
```
- 2 Check nothing is left uncommitted:

```
git status
```

Anything listed as modified or untracked will **not** reach the Pi.
- 3 Commit, then push the branch to GitHub yourself.
- 4 Write down the exact branch name you pushed – you will type it during the clone. (Today that is `Franklin-vibe-fable`.)

LOOK OUT FOR

`git clone` copies only what is on GitHub. The single most common "the Pi is missing my feature" cause is work that was never committed or never pushed from the laptop. Also note the old setup notes name the `Franklin's-vibe-coding` branch – the apostrophe in that name must be quoted in the shell, which is one more reason the newer branch name is easier to work with.

YOUR LAPTOP'S TEST DATA STAYS BEHIND — ON PURPOSE

`data/kiosk.db` is not in git, so the Pi starts with a fresh database and seeds the demo printers on first run. That is usually what you want for a clean launch. If you *do* want to carry the laptop's data over, copy the file once after everything is installed: stop the service,

```
scp data/kiosk.db pi@raspberrypi.local:/home/pi/maintenance_kiosk/data/
```

start the service.

3 Install on the Pi

Five recipes, in order. Run them over SSH or with a keyboard on the Pi.

3.1 Prepare Raspberry Pi OS

- 1 Use **Raspberry Pi OS 64-bit** (the 32-bit image can miss pre-built Python packages).
- 2 Install the system pieces:

```
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip chromium-browser
```

- 3 Set the timezone *before* anything else:

```
sudo timedatectl set-timezone Europe/London
timedatectl
```

LOOK OUT FOR

The Monday-12:00 reminder timer fires at **Pi local time**. A Pi left on UTC sends reminders an hour off for half the year – and nobody notices until someone asks why the email came at 11. Check `timedatectl` says `Europe/London` and that NTP sync is active.

3.2 Get the code

- 1 Clone the branch you pushed:

```
cd /home/pi
git clone -b Franklin-vibe-fable https://github.com/iforge-uos/maintenance_kiosk
cd /home/pi/maintenance_kiosk
```

LOOK OUT FOR

If the repository is private, the Pi needs its own way to log in to GitHub – a fine-grained personal access token pasted at the clone prompt is the simplest (username = your GitHub name, password = the token). Never store your real GitHub password on the kiosk. The path matters too: the systemd templates assume exactly

`/home/pi/maintenance_kiosk`.

3.3 Python environment

1

```
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install -r requirements.txt
```

This includes `gunicorn`, the production web server. First install on a Pi can take a few minutes – that's normal.

3.4 Create the .env

1

```
cp .env.example .env
nano .env
```

2

Fill in only what belongs there (see the table below). Leave `DATABASE_URL=` **blank**.

Since the Settings-page rework, `.env` is small. Most configuration now lives in the kiosk's own Settings page (stored in the database), where the team can change it without touching the Pi:

SETTING	WHERE IT LIVES NOW
Email sending (<code>SMTP_HOST/PORT/USERNAME/PASSWORD/FROM_EMAIL</code>)	<code>.env</code> – use a Gmail app password , never the real account password
<code>DATABASE_URL</code>	<code>.env</code> – leave blank for SQLite
<code>GOOGLE_CHAT_WEBHOOK_SECRET</code>	<code>.env</code> – only if you use the Apps Script relay mode; otherwise blank
Discord webhook URL	Settings page
Google Chat webhook / relay URL	Settings page
Google Sheets sync on/off, URL, secret	Settings page (the old <code>.env</code> values still work as a fallback)
Kiosk base URL, printers, technicians, semesters, admin PIN	Settings page

LOOK OUT FOR

- A filled-in `DATABASE_URL` makes the app try to reach PostgreSQL and die at boot. This is the number-one "service won't start" cause.
- The `.env` file must **exist** even if you send no email – the systemd units declare `EnvironmentFile=.../.env` and refuse to start without the file.
`cp .env.example .env` is enough.
- No `FLASK_SECRET_KEY` needed: the app generates one on first run and stores it in the database.

3.5 First run & smoke test

- 1 Create and seed the database, then run the test suite:

```
.venv/bin/python -c "from app import initialize_database; initialize_data  
.venv/bin/python -m unittest discover -s tests
```

- 2 Start the web app by hand once:

```
.venv/bin/gunicorn --bind 0.0.0.0:5050 app:app
```

- 3 Open `http://raspberrypi.local:5050` from your laptop. You should see the dashboard with demo printers. `Ctrl-C` stops it again.

LOOK OUT FOR

Test with **gunicorn**, not `python app.py` – the dev server binds to 127.0.0.1 only, so "I can't reach it from my laptop" during this step usually means you started the wrong one. Gunicorn binds `0.0.0.0` and is reachable from the whole workshop network – which also means anyone on that network can open the kiosk. That's by design; the admin PIN (next section) is what protects Settings and the Diagnosis Editor.

4 First configuration on the kiosk

Five minutes in the Settings page while gunicorn is still running by hand.

Settings page, top to bottom

- 1 Technicians** – real names and emails; tick "Receives weekly reminders" only for people who want the Monday email.
- 2 Printers** – rename/reorder to match the real farm.
- 3 Notifications** – paste the **Discord webhook URL** (Discord server → Server Settings → Integrations → Webhooks → New Webhook → copy URL). Google Chat too, if used.
- 4 Kiosk base URL** – set to `http://raspberrypi.local:5050` (or the Pi's fixed IP). This is the link inside reminder emails.
- 5 Google Sheets sync** – if you use it: tick the box, paste the Apps Script URL and secret (full Sheets/Apps-Script setup is in `docs/RASPBERRY_PI_SETUP.md`, sections 6-7), then run one backfill:

```
.venv/bin/python google_sheets_sync_worker.py --backfill
```
- 6 Security** – **set the admin PIN now**. Until you do, anyone at the kiosk (or on the network) can open Settings and the Diagnosis Editor.

LOOK OUT FOR

A kiosk base URL left at `127.0.0.1` makes every reminder email link open nothing on people's phones – it's the single most-reported "broken email" that isn't actually broken email. And write the admin PIN down somewhere sensible: resetting a forgotten PIN needs shell access (the exact command is in section 9).

5 Make it permanent — systemd

After this, the kiosk survives reboots, crashes, and power cuts.

The repo ships ready-made unit files in `deploy/systemd/`: one service for the web app, plus three timer/service pairs for the workers (notifications every 5 min, sheets sync every 5 min, weekly reminder Monday 12:00).

Install and switch on

- 1 Copy the units and reload systemd:

```
sudo cp deploy/systemd/maintenance-kiosk*.service /etc/systemd/system/  
sudo cp deploy/systemd/maintenance-kiosk*.timer /etc/systemd/system/  
sudo systemctl daemon-reload
```

- 2 Enable everything to start now *and* at every boot:

```
sudo systemctl enable --now maintenance-kiosk.service  
sudo systemctl enable --now maintenance-kiosk-notifications.timer  
sudo systemctl enable --now maintenance-kiosk-sheets-sync.timer  
sudo systemctl enable --now maintenance-kiosk-weekly-reminder.timer
```

- 3 Verify:

```
systemctl status maintenance-kiosk.service           # wants: active (running)  
systemctl list-timers "maintenance-kiosk*"           # wants: 3 timers with
```

LOOK OUT FOR

- The templates hard-code `User=pi`, `Group=pi` and `/home/pi/maintenance_kiosk` everywhere. A Pi with a different username needs those lines edited in **all seven files** (`User=`, `Group=`, `WorkingDirectory=`, `EnvironmentFile=`, `ExecStart=`) *before* copying.
- After editing a unit that is already installed, re-copy it and run `sudo systemctl daemon-reload` again – systemd reads the copy in `/etc/systemd/system/`, not your repo.
- The timers are `Persistent=true`: if the Pi was off at Monday 12:00, the reminder fires once at the next boot. Don't be surprised by a "late" reminder email after a power cut.

6 The touchscreen

Full-screen Chromium, no cursor, no screen blanking.

Kiosk mode on X11 (LXDE — the classic Pi desktop)

1

```
mkdir -p ~/.config/lxsession/LXDE-pi
nano ~/.config/lxsession/LXDE-pi/autostart
```

2

Put exactly this in the file:

```
@xset s off
@xset -dpms
@xset s noblank
@unclutter -idle 0.5
@chromium-browser --kiosk --app=http://127.0.0.1:5050
```

3

`sudo reboot` – the dashboard should appear full-screen on its own.

LOOK OUT FOR

- **Newer Raspberry Pi OS (Bookworm) boots into Wayland, where this autostart file does nothing.** Quickest fix: `sudo raspi-config` → Advanced Options → Wayland → select **X11**, reboot, then the recipe above works as written. (Staying on Wayland is possible – the autostart lines go into the compositor's own config instead – but X11 keeps this setup simple.)
- The three `@xset` lines are what stop the screen going black after 10 minutes; if the screen still blanks, disable screen blanking in `raspi-config` → Display Options as well.
- `--app=` (not just `--kiosk`) hides the address bar so visitors can't type URLs on the touchscreen.

Launch checklist — prove the whole chain once

- 1 Reboot the Pi. The dashboard opens full-screen by itself.
- 2 Trigger a test fault on a printer (Reactive Start → manual) – the card turns red immediately.
- 3 Wait up to 5 minutes → the "fault started" message appears in Discord.
- 4 Mark the fault fixed → within 5 minutes the "fault fixed" message follows.
- 5 If Sheets sync is on: submit one weekly record, then check the spreadsheet gained a row.
- 6 Check the timers show next-run times: `systemctl list-timers "maintenance-kiosk*"`.

REMEMBER THE 5-MINUTE RULE

Discord/Chat/email messages are *queued* at the moment of the save and *sent* by the timer worker – up to five minutes later. A message that hasn't arrived after one minute is not broken; a message missing after ten is worth investigating (section 9).

7 Updating the Pi later

Redeploys are two commands – plus one check that saves the diagnosis tree.

Redeploy new code

- 1 **First, the check:** has anyone edited the Diagnosis Editor on the Pi since the last update?

```
cd /home/pi/maintenance_kiosk
git status
```

If `data/diagnosis_tree.json` shows as *modified*, keep a copy before pulling:

```
cp data/diagnosis_tree.json /home/pi/tree-pi-edits-$(date +%Y%m%d).json
```

- 2 Pull and restart:

```
git pull
sudo systemctl restart maintenance-kiosk.service
```

The worker timers pick up new code automatically at their next run – only the web service needs the restart.

- 3 If the pull refused because of the tree file: your safety copy holds the Pi's edits. Either commit the Pi's version (`git add data/diagnosis_tree.json && git commit`) before pulling again, or pull with `git checkout -- data/diagnosis_tree.json && git pull` and re-apply the Pi's edits from the copy via the Raw JSON editor.

THE ONE REAL TRAP IN THIS WHOLE SETUP

`data/diagnosis_tree.json` is **tracked by git**, and the Diagnosis Editor writes to it. That means tree edits made on the Pi and tree edits made on the laptop can collide at `git pull` time. Decide as a team where tree editing happens (recommendation: **on the Pi**, since that's where the assistant runs, with the Pi's file committed back whenever someone updates code). The database and `.env` have no such problem – git ignores them completely.

Backups

Back up the three things that matter (do this before every update, and monthly)

1

```
sudo systemctl stop maintenance-kiosk.service
mkdir -p /home/pi/kiosk-backups/$(date +%Y%m%d)
cp data/kiosk.db data/diagnosis_tree.json /home/pi/kiosk-backups/$(date +%Y%m%d)
cp -r data/diagnosis_tree_backups /home/pi/kiosk-backups/$(date +%Y%m%d)/
sudo systemctl start maintenance-kiosk.service
```

2

Now and then, copy that folder off the Pi entirely (`scp` to a laptop or a USB stick) – an SD card is the least reliable part of a Raspberry Pi.

LOOK OUT FOR

Stop the service before copying `kiosk.db` – copying a SQLite file while it's being written can produce a corrupt backup that only fails when you need it.

8 When something looks wrong

Symptom → first command → usual cause.

SYMPTOM	RUN THIS FIRST	USUAL CAUSE
Web app won't start	<code>journalctl -u maintenance-kiosk.service -n 100 --no-pager</code>	Filled-in <code>DATABASE_URL</code> ; missing <code>.env</code> ; wrong paths/username in the unit files; venv not created
Dashboard unreachable from other devices	<code>systemctl status maintenance-kiosk</code> , then try the Pi's IP instead of <code>raspberrypi.local</code>	App started with <code>python app.py</code> (binds 127.0.0.1) instead of the service; or mDNS name not resolving on that device
No Discord / Chat message	<code>.venv/bin/python notification_worker.py</code> and read its output	Waited less than 5 min; webhook URL wrong in Settings; Pi offline – rows wait in the queue and retry on their own
Monday reminder never arrived	<code>timedatectl</code> · <code>systemctl list-timers "maintenance-kiosk-weekly"</code>	Wrong timezone; timer not enabled; SMTP credentials wrong (run <code>notification_worker.py</code> and read the error); technician's reminder box unticked
Google Sheet not updating	<code>.venv/bin/python google_sheets_sync_worker.py --limit 10</code>	Sync unticked in Settings; secret mismatch with Apps Script; rows are queued in <code>sync_queue</code> and will catch up once fixed
Screen goes black after minutes	re-check the autostart file's <code>@xset</code> lines	Autostart file in the wrong place; Pi booted into Wayland (section 6); blanking also set in <code>raspi-config</code>
Forgot the admin PIN	<code>sqlite3 data/kiosk.db "DELETE FROM app_settings WHERE setting_key='admin_pin_hash';"</code> then restart the service	The PIN is stored hashed and can't be recovered – deleting the row unlocks Settings so a new PIN can be set immediately
Everything broken after an update	<code>git log --oneline -3</code> · <code>journalctl -u maintenance-kiosk -n 100</code>	Roll back with <code>git checkout <previous-commit></code> + restart, then debug on the laptop, not on the Pi

THE GOLDEN DEBUGGING ORDER

1) `journalctl` for the service involved → 2) run the worker by hand and read what it prints → 3) the kiosk's own log diary at `/logs` → 4) only then start editing config. Saves are never lost while you debug – everything waits in the queues.

Pi Shipping Manual · September 2026 · companion to the Maintenance Kiosk Team
Manual · deeper Google Sheets / Google Chat setup lives in

`docs/RASPBERRY_PI_SETUP.md`).